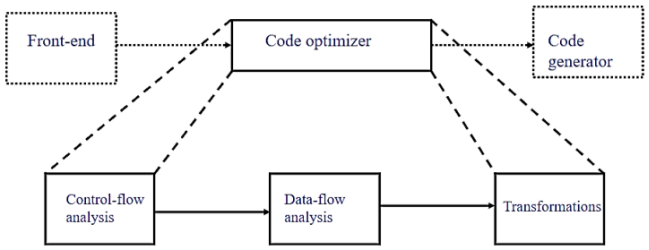
UNIT -5

Data Flow Analysis



Data flow analysis is a global code optimization technique. The compiler performs code optimization efficiently by collecting all the information about a program and distributing it to each block of its **control flow graph (CFG)**. This process is known as data flow analysis.

Since optimization has to be done on the entire program, the whole program is examined, and the compiler collects information.

The control flow graph represents a program in blocks depicting how the control of the program is being passed from one block to another.

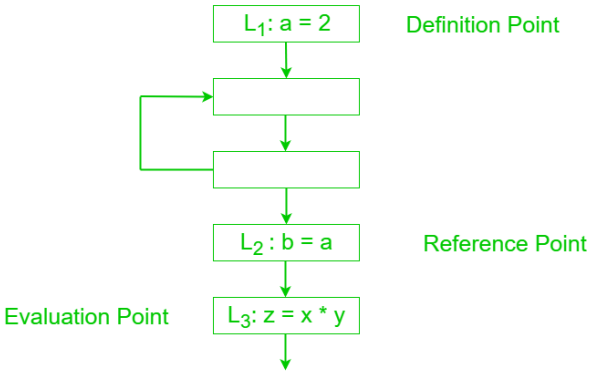
➢ Points And Paths

A "point" in data flow analysis refers to a specific location within a program's control flow graph (CFG) where information about the program's data state is collected and analyzed. These points are crucial for understanding how data values change and are used throughout the program, enabling various code optimizations.

Within a basic block, we talk of the point between two adjacent statements, as well as the point before the first statement and after the last. Thus, block B1 has four points if blcok has 3 assignments. one before any of the assignments and one after each of the three assignments.

Below are some basic terminologies related to data flow analysis.

- **Definition Point-** A definition point is a point in a program that defines a data item.
- **Reference Point-** A reference point is a point in a program that contains a reference to a data item.
- **Evaluation Point-** An evaluation point is a point in a program that contains an expression to be evaluated.



The diagram shows an example of a definition point, a reference point, and an evaluation point in a program.

A "**path**" in data flow analysis represents a sequence of program points that could be traversed during program execution.

A path from p1 to p_n is a sequence of points p1, p2,….,pn such that for each i between 1 and n-1, either

1. P_i is the point immediately preceding a statement and P_{i+1} is the point immediately following that statement in the same block, or
2. P_i is the end of some block and P_{i+1} is the beginning of a successor block.

➢ Forward And Backward Analysis In Data Flow Analysis

In compiler design, forward and backward analysis are classified based on the direction in which data-flow information is propagated through the control-flow graph (CFG).

Forward Analysis	**Backward Analysis**
What information reaches this point from the past?	What information is needed at this point for the future?
Direction: Entry → Exit	Direction: Exit → Entry
Information comes from: Predecessor blocks	Information comes from: Successor blocks
Examples : Reaching Definitions, Available Expressions, Constant Propagation	Examples : Live Variable Analysis, Very Busy Expressions, Dead Code Elimination

General Data Flow Analysis Schema

All these analyses can be expressed using a data flow framework with:

Direction: Forward or Backward

Domain: Set of data flow values (e.g., sets of variables, expressions)

Meet operator: Union or Intersection

Transfer function: Defines how information changes across a statement or block

➢ Data Flow values

Data flow values are the information propagated between basic blocks during data flow analysis. They help the compiler determine properties such as which definitions reach a point, which variables are live, and which expressions are available.

Types of data flow values

USE(B)

USE(B) contains only variables that are used before they are defined within the same basic block. Consider

1. x = p + q
2. y = x + r USE(B) = {p, q, r}

DEF(B)

DEF(B) is the set of variables that are assigned values within a basic block B. or calculate DEF(B), scan every statement and collect the variables on the left-hand side of assignment statements. Include each variable only once.

Example :
1. x = p + q
2. y = x + r
3. p = y + q
4. x = y + 5 DEF(B) = {x, y, p}

GEN(B)

GEN(B) is the set of definitions generated by a basic block that reach the end of that block without being overwritten by later assignments within the same block.

Example 1

d1: x = p + q
d2: y = x + r
d3: p = y + q
d4: x = y + 5

GEN(B) = { d2, d3, d4}

Example 2

d1: a = 10
d2: b = 20
d3: a = b + 5
d4: c = a + b
d5: b = 30

GEN(B) = { d3, d4, d5}

KILL(B)

KILL(B) contains all definitions outside B that assign to variables defined within B, regardless of whether those definitions are in preceding, succeeding, or unrelated blocks.

Example:

Consider the basic block B

d1: a = 10
d2: b = 20
d3: a = b + 5
d4: c = a + b
d5: b = 30

Suppose other blocks in the program contain these definitions:

d6: a = 50
d7: b = 60
d8: c = 70
d9: d = 80

KILL(B) = { d6,d7,d8 }

GEN/KILL are commonly used in Reaching Definitions and Available Expressions. USE/DEF are commonly used in Live Variable Analysis. Very Busy Expressions generally use expression-based GEN/KILL sets.

➢ Data Flow Analysis Types / Properties

Data flow analysis is commonly classified into four major types: reaching definitions, live variable analysis, available expressions, and very busy expressions. Each analysis propagates specific information through the control flow graph to support compiler optimization.

Some properties of the data flow analysis are-

1. Available expression
2. Reaching definition
2. Live variable
4. Very Busy expression

a) Available Expression

An expression **a + b** is said to be available at a program point **x** if none of its operands gets modified before their use. It is used to eliminate *common subexpressions*.

- It is a forward data-flow analysis.
- It identifies expressions that have already been computed and whose operands have not changed.
- It uses the intersection operation.
- It is a **must** analysis, because an expression must be available along every incoming path.
- It helps eliminate redundant computations through common subexpression elimination.
- An expression becomes unavailable if any of its operands is redefined.

Example:

1: a = b + c
2: d = a * 2
3: e = b + c

After line 1, expression b + c is available (already computed and variables b, c unchanged).
So at line 3, compiler can replace b + c with previously computed value → optimization by common subexpression elimination

Example

1. x = a + b
2. y = c * d
3. a = 10
4. z = c * d

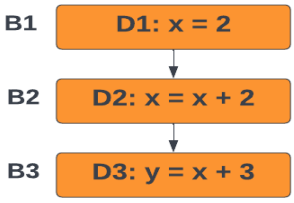
Statement	Explanation	Available expressions after statement
1. x = a + b	Compute a + b	{a + b}
2. y = c * d	Compute c * d	{a + b, c * d}
3. a = 10	a + b is invalidated because a changes	{c * d}
4. z = c * d	c * d is still available; it can be reused	{c * d}

b) Reaching Definition

A definition of a variable is said to reach a program point x if there is a path from that definition to x along which the variable is not redefined.

- It is a forward data-flow analysis.
- It identifies definitions that may reach a particular program point without being overwritten.
- It uses the union (U) operation.
- It is a may analysis, because a definition may reach a program point along at least one incoming path.
- It uses GEN and KILL sets to determine which definitions are generated and which are invalidated.
- It helps the compiler track variable definitions and supports optimizations such as constant propagation, Variable Propagation, Undefined Variable Detection
- A definition becomes invalid along a path if the same variable is redefined before reaching the program point.

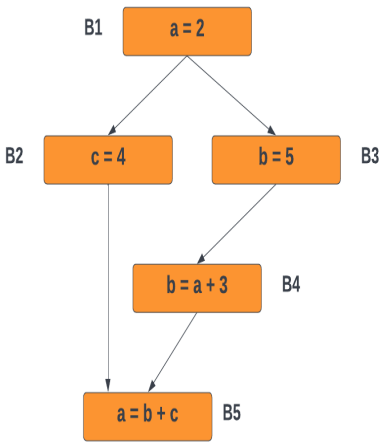
Example:



In the above example, D1 is a reaching definition for block B2 since the value of x is not changed (it is two only) but D1 is not a reaching definition for block B3 because the value of x is changed to x + 2. This means D1 is killed or redefined by D2.

c) Live Variable

A variable is said to be live at a program point x if its current value may be used in the future before the variable is redefined.



- It is a backward data-flow analysis.
- It identifies variables whose current values may be needed in subsequent statements.
- It uses the union (U) operation.
- It is a may analysis, because a variable is considered live if its value may be used along at least one future path.
- It uses USE and DEF sets to determine which variables are used and which are defined in a basic block.
- It helps eliminate dead code and supports efficient register allocation.
- A variable becomes dead if its current value is not used before the variable is redefined or the program terminates.

In the above example, the variable 'a' is live at blocks B1,B2, B3 and B4 but is killed at block B5 since its value is changed from 2 to b + c. Similarly, variable b is live at block B3 but is killed at block B4.

d) Very Busy Expression

An expression is said to be busy along a path if its evaluation occurs along that path, but none of its operand definitions appears before it.

- It is a backward data-flow analysis.
- It identifies expressions that are guaranteed to be evaluated later before any of their operands are redefined.
- It uses the intersection operation.
- It is a must analysis, because an expression must be evaluated along every possible execution path.
- It helps in compiler optimization through code hoisting, where computations may be moved earlier when it is safe.
- An expression is not very busy if an operand is redefined before the expression is evaluated.

Example :

1. x = 10
2. y = a + b
3. z = c * d
4. a = 20

Statement	Explanation	Very busy expressions at entry
4. `a = 20`	Expressions involving `a` cannot be carried backward through this assignment	`{}`
3. `z = c *d`	`c * d` is evaluated here	`{c * d}`
2. `y = a +b`	`a + b` is evaluated here	`{a + b, c * d}`
1. `x = 10`	Neither `a`, `b`, `c`, nor `d` changes	`{a + b, c * d}`

The nature of data flow values depends on the specific data flow analysis being performed.

In data flow analysis, the **meet operator** is used to combine information from different control flow paths that converge at a single program point. It determines a single, conservative value for that point

➢ Data Flow Equations

Data flow equations describe how information enters, leaves, and changes within each basic block of a program. They are used by the compiler to analyze program behavior and perform code optimization.

Analysis Type	Direction	Key Sets	IN/OUT Equations	Used For
Available Expressions	Forward	GEN, KILL	$IN[B] = \bigcap_{P \in pred(B)} OUT[P]$ $OUT[B] = GEN[B] \cup (IN[B] - KILL[B])$	Common subexpression elimination
Reaching Definitions	Forward	GEN, KILL	$IN[B] = \bigcup_{P \in pred(B)} OUT[P]$ $OUT[B] = GEN[B] \cup (IN[B] - KILL[B])$	Constant propagation and related optimizations
Live Variables	Backward	USE, DEF	$OUT[B] = \bigcup_{S \in succ(B)} IN[S]$ $IN[B] = USE[B] \cup (OUT[B] - DEF[B])$	Register allocation and dead code elimination
Very Busy Expressions	Backward	GEN, KILL	$OUT[B] = \bigcap_{S \in succ(B)} IN[S]$ $IN[B] = GEN[B] \cup (OUT[B] - KILL[B])$	Code motion and redundancy removal

In Data Flow Analysis, IN(B) and OUT(B) represent the information entering and leaving a basic block \(B\).

- IN(B): Set of data-flow information available at the entry of basic block \(B\).
- OUT(B): Set of data-flow information available at the exit of basic block \(B\).

Example for Reaching Definition analysis

Forward Analysis – Reaching Definitions

1. Program and basic blocks

B1	B2	B3
$d1$: a = 10 $d2$: x = 5	$d3$: b = a + 2 $d4$: a = 20	$d5$: c = b + 3 print(c)

2. Data-flow equations (Forward)

$$IN(B) = \bigcup_{P \in Pred(B)} OUT(P)$$
$$OUT(B) = GEN(B) \cup (IN(B) - KILL(B))$$

3. GEN and KILL sets

Block	GEN(B)	KILL(B)	Explanation
B1	$\{d1, d2\}$	$\{d4\}$	Generates d_1, d_2. Kills d_4 (redefines a).
B2	$\{d_3, d4\}$	$\{d1\}$	Generates d_3, d_4. Kills d_1 (redefines a).
B3	$\{d5\}$	$\emptyset$	Generates d_5. No redefinition of c.

4. Step-by-step computation of IN and OUT

Block	IN(B)	Calculation of OUT(B)	OUT(B)
B1	$\emptyset$	$OUT(B_1) = \{d_1, d_2\} \cup (\emptyset - \{d4\})$ $= \{d_1, d2\} \cup \emptyset$ $= \{d_1, d2\}$	$\{d1, d2\}$
B2	$\{d1, d2\}$	$OUT(B_2) = \{d_3, d4\} \cup (\{d_1, d_2\} - \{d_1\})$ $= \{d_3, d4\} \cup \{d_2\}$ $= \{d_2, d_3, d4\}$	$\{d_2, d_3, d4\}$
B3	$\{d_2, d_3, d4\}$	$OUT(B_3) = \{d_5\} \cup (\{d_2, d_3, d4\} - \emptyset)$ $= \{d_5\} \cup \{d_2, d_3, d4\}$ $= \{d_2, d_3, d4, d5\}$	$\{d_2, d_3, d4, d_5\}$

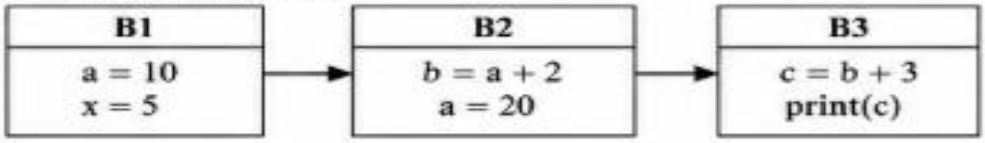
5. Final result (Forward Analysis)

Block	GEN(B)	KILL(B)	IN(B)	OUT(B)
B1	$\{d1, d2\}$	$\{d4\}$	$\emptyset$	$\{d1, d2\}$
B2	$\{d_3, d4\}$	$\{d1\}$	$\{d1, d2\}$	$\{d_2, d_3, d4\}$
B3	$\{d5\}$	$\emptyset$	$\{d_2, d_3, d4\}$	$\{d_2, d_3, d4, d5\}$

Example for Live variable analysis

Backward Analysis – Live Variable Analysis

1. Program and basic blocks



2. Data-flow equations (Backward)

$$OUT(B) = \bigcup_{S \in Succ(B)} IN(S)$$

$$IN(B) = USE(B) \cup (OUT(B) - DEF(B))$$

3. USE and DEF sets

Block	USE(B)	DEF(B)	Explanation
B1	$\emptyset$	$\{a, x\}$	Both a and x are defined; no use before definition.
B2	$\{a\}$	$\{a, b\}$	a is used in $b = a + 2$; a and b are defined.
B3	$\{b\}$	$\{c\}$	b is used in $c = b + 3$; c is defined (print(c) uses c but it is already defined in this block).

4. Step-by-step computation of IN and OUT

Block	OUT(B)	Calculation of IN(B)	IN(B)
B3	$\emptyset$	$IN(B_3) = \{b\} \cup (\emptyset - \{c\}) = \{b\} \cup \emptyset = \{b\}$	$\{b\}$
B2	$\{b\}$	$IN(B_2) = \{a\} \cup (\{b\} - \{a, b\}) = \{a\} \cup \emptyset = \{a\}$	$\{a\}$
B1	$\{a\}$	$IN(B_1) = \emptyset \cup (\{a\} - \{a, x\}) = \emptyset \cup \emptyset = \emptyset$	$\emptyset$

5. Final result (Backward Analysis)

Block	USE(B)	DEF(B)	IN(B)	OUT(B)
B1	$\emptyset$	$\{a, x\}$	$\emptyset$	$\{a\}$
B2	$\{a\}$	$\{a, b\}$	$\{a\}$	$\{b\}$
B3	$\{b\}$	$\{c\}$	$\{b\}$	$\emptyset$

➢ Partial Redundancy Elimination in Compiler Design

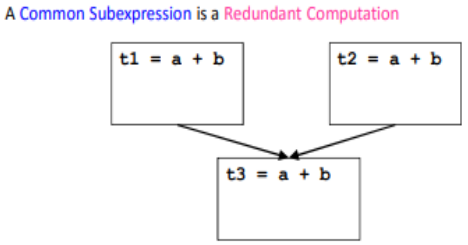
PRE (Partial Redundancy Elimination) is a compiler optimization method.

Partial Redundancy Elimination (PRE) is a compiler optimization technique that eliminates computations that are redundant on some execution paths but not on all paths.

In simple words, an expression is partially redundant when it has already been computed along some paths, but not along others.

An example of full redundancy is given below.

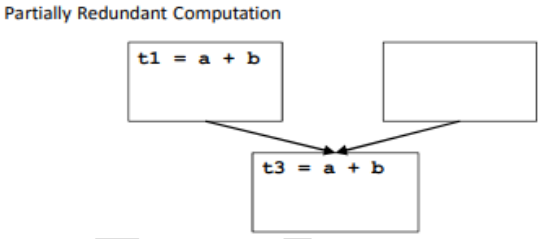
```
if(condition1) {
----
----
x = a + b;
}
else{
x = a + b;
-----
----
}
```



As you can see in the above example, the computation of a+b is fully redundant, as it is performed along both execution paths.

Example of partial redundancy is given below.

```
if (condition) {
   x = a + b;
}
else {
   y = 10;
}
z = a + b;
```



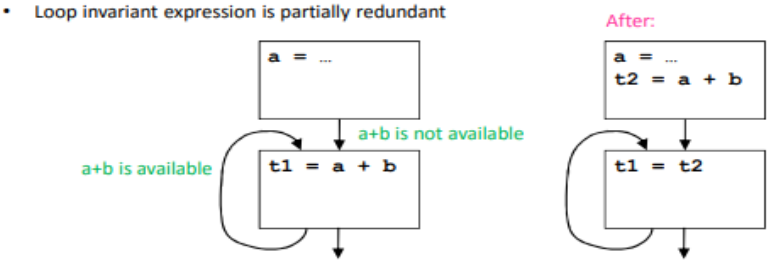
There are two possible execution paths:

- True path: a + b is computed inside the if block and then computed again at z = a + b. The second computation is redundant, assuming a and b do not change.
- False path: a + b has not been computed earlier, so the computation at z = a + b is necessary.

Therefore, a + b is redundant on only one path. This is called partial redundancy.

The compiler can move the computation to before the branch, making the expression available along both paths. After optimization

```
t = a + b;
if (condition) {
     x = t;
}
else {
     y = 10;
}
z = t;
```



Redundancy is due to the presence of one or multiple expressions from the following expressions.

1. Partially redundant expressions:
   It is an expression in which the value that is computed by the expression is already available on some of the paths but is not available on all the paths through a program to that expression.
2. Loop invariant expression:
   It is a condition that is necessarily true immediately before and immediately after each iteration of the loop.

3. Common sub-expression:

A common sub-expression is an expression that has appeared and computed before and appears again during the computation of the code.

To implement PRE in modern compilers. Some of the algorithms that are used worldwide are:

Lazy Code Motion Algorithm (LCM)

Global Value Numbering Algorithm (GVN).

➢ LOOPS IN FLOW GRAPH

A loop in a flow graph is a path in which control returns to an already visited basic block, allowing a set of statements to execute repeatedly.

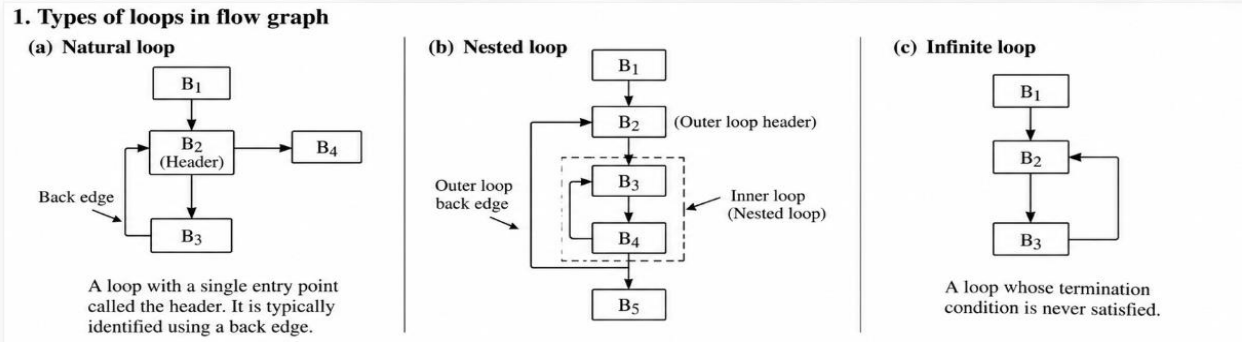
Loops are important in compiler optimization because improving code inside a loop can significantly reduce execution time.

Types of loops

1. Natural loop: A loop with a single entry point called the header. It is typically identified using a back edge.

2. Nested loop: A loop that exists entirely inside another loop.

3. Infinite loop: A loop whose termination condition is never satisfied.



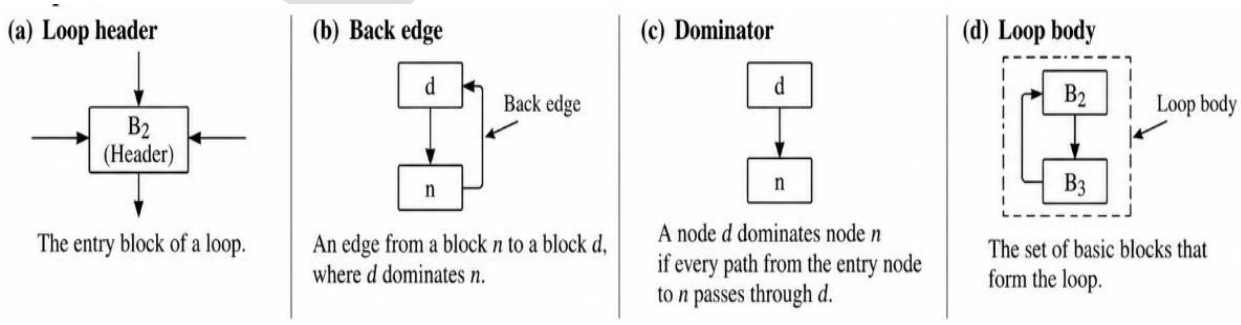
Loop header: The entry block of a loop.

Back edge: An edge from a block n to a block d, where d dominates n.

Dominator:

A node *d* dominates node *n* if every path from the entry node to *n* passes through *d*. Every initial node dominates all the remaining nodes in the flow graph and the entry of a loop dominates all nodes in the loop. Similarly every node dominates itself.

Loop body: The set of basic blocks that form the loop.



Example:

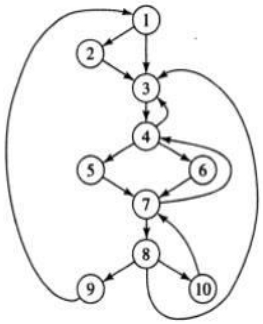
In the flow graph below,

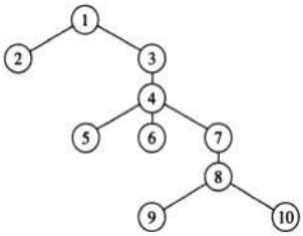
Initial node,node1 dominates every node.

Node 2 dominates itself

Node 3 dominates all but 1 and 2. Node 4 dominates all but 1,2 and 3.

Node 5 and 6 dominates only themselves, since flow of control can skip around either by goin through the other. Node 7 dominates 7,8 ,9 and 10. Node 8 dominates 8,9 and 10. Node 9 and 10 dominates only themselves.





The way of presenting dominator information is in a tree, called the **dominator tree**, in which

- The initial node is the root.
- The parent of each other node is its immediate dominator.
- Each node d dominates only its descendents in the tree.

➢ **Reducible Flow Graphs:**

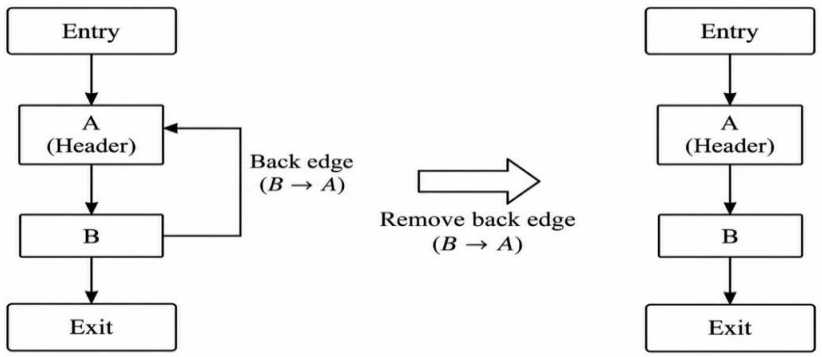
Reducible flow graphs are special flow graphs, for which several code optimization transformations are especially easy to perform, loops are unambiguously defined, dominators can be easily calculated, data flow analysis problems can also be solved efficiently. Exclusive use of structured flow-of-control statements such as if-then-else, while-do, continue, and break statements produces programs whose flow graphs are always reducible.

The most important properties of reducible flow graphs are that

1. There are no umps into the middle of loops from outside;
2. The only entry to a loop is through its header

Definition:

A flow graph is called a reducible flow graph if its edges can be partitioned into forward edges and back edges, such that the forward edges form an acyclic graph and every back edge points to a node that dominates its source.



(a) Original flow graph (reducible) **(b) Reduced graph (acyclic)**

Here, `A` is the loop header. The edge B→ A is a back edge because `A` dominates `B`. Removing this back edge leaves an acyclic graph, so the graph is reducible.

For reducible flow graph analysis, we temporarily remove back edges to check that the remaining graph is acyclic (contains no cycles).

- The loop is not removed from the actual program.
- Only the edge is removed from the graph for this particular analysis step.
- The resulting graph is used to reason about the graph's structure.